JavaScript 设计模式

JavaScript 是一门多范式编程语言，设计模式在 JS 中有独特的实现方式。

一、创建型模式 (Creational Patterns)

1. 单例模式 (Singleton)

```
const Singleton = (function() {
    let instance = null;

    function createInstance() {
        return {
            name: 'Singleton',
            method() {
                console.log('单例方法');
            }
        };
    }

    return {
        getInstance() {
            if (!instance) {
                instance = createInstance();
            }
            return instance;
        }
    };
})();

const obj1 = Singleton.getInstance();
const obj2 = Singleton.getInstance();
console.log(obj1 === obj2); // true
```

2. 工厂模式 (Factory)

```
class Car {
    constructor(options) {
        this.doors = options.doors || 4;
        this.state = options.state || 'new';
    }
}

class Truck {
    constructor(options) {
        this.doors = options.doors || 2;
        this.state = options.state || 'used';
    }
}

class VehicleFactory {
```

```
    createVehicle(options) {
        if (options.type === 'car') {
            return new Car(options);
        } else {
            return new Truck(options);
        }
    }
}

const factory = new VehicleFactory();
const car = factory.createVehicle({ type: 'car', doors: 4 });
```

3. 建造者模式 (Builder)

```
class UserBuilder {
    constructor(name) {
        this.user = { name };
    }

    setAge(age) {
        this.user.age = age;
        return this;
    }

    setEmail(email) {
        this.user.email = email;
        return this;
    }

    build() {
        return this.user;
    }
}

const user = new UserBuilder('张三')
    .setAge(25)
    .setEmail('zhangsan@example.com')
    .build();
```

4. 原型模式 (Prototype)

```
const carProto = {
    drive() {
        console.log('Driving', this.brand);
    }
};

const car1 = Object.create(carProto);
car1.brand = 'Toyota';

const car2 = Object.create(carProto);
car2.brand = 'Honda';
```

二、结构型模式 (Structural Patterns)

1. 适配器模式 (Adapter)

```
class OldPayment {
    payWithCash() {
        console.log('现金支付');
    }
}

class PaymentAdapter {
    constructor() {
        this.oldPayment = new OldPayment();
    }

    pay(amount) {
        this.oldPayment.payWithCash();
        console.log('金额:', amount);
    }
}

const adapter = new PaymentAdapter();
adapter.pay(100);
```

2. 装饰器模式 (Decorator)

```
function Coffee() {
    this.cost = function() { return 5; };
}

function Milk(coffee) {
    this.cost = function() {
        return coffee.cost() + 2;
    };
}

function Sugar(coffee) {
    this.cost = function() {
        return coffee.cost() + 1;
    };
}

let coffee = new Coffee();
coffee = new Milk(coffee);
coffee = new Sugar(coffee);
console.log(coffee.cost()); // 8
```

3. 代理模式 (Proxy)

```
const realSubject = {
    request() {
        console.log('真实请求');
    }
};

const proxy = new Proxy(realSubject, {
    apply(target, context, args) {
```

```
        console.log('代理预处理');
        target.request();
        console.log('代理后处理');
    }
});

proxy.request();
```

4. 组合模式 (Composite)

```
class Component {
    constructor(name) {
        this.name = name;
    }

    add(component) {}
    remove(component) {}
    display(depth) {}
}

class Leaf extends Component {
    display(depth) {
        console.log('-'.repeat(depth) + this.name);
    }
}

class Composite extends Component {
    constructor(name) {
        super(name);
        this.children = [];
    }

    add(component) {
        this.children.push(component);
    }

    display(depth) {
        console.log('-'.repeat(depth) + this.name);
        this.children.forEach(child => child.display(depth + 2));
    }
}
```

5. 外观模式 (Facade)

```
const Mortgage = (function() {
    function Mortgage(name) {
        this.name = name;
    }

    function bank() { console.log('检查银行...'); }
    function credit() { console.log('检查信用...'); }
    function background() { console.log('背景调查...'); }

    Mortgage.prototype.apply = function(amount) {
        let result = true;
        bank();
```

```
        credit();
        background();
        console.log(`${this.name} 贷款 ${amount} 结果: ${result}`);
        return result;
    };

    return Mortgage;
})();

new Mortgage('张三').apply(500000);
```

三、行为型模式 (Behavioral Patterns)

1. 观察者模式 (Observer)

```
class EventEmitter {
    constructor() {
        this.events = {};
    }

    on(event, listener) {
        if (!this.events[event]) {
            this.events[event] = [];
        }
        this.events[event].push(listener);
    }

    emit(event, data) {
        if (this.events[event]) {
            this.events[event].forEach(listener => listener(data));
        }
    }

    off(event, listenerToRemove) {
        if (this.events[event]) {
            this.events[event] = this.events[event]
                .filter(listener => listener !== listenerToRemove);
        }
    }
}

const emitter = new EventEmitter();
emitter.on('message', (data) => console.log('收到:', data));
emitter.emit('message', 'Hello World');
```

2. 策略模式 (Strategy)

```
class PaymentContext {
    constructor(strategy) {
        this.strategy = strategy;
    }

    pay(amount) {
        this.strategy.pay(amount);
```

```
    }
}

class CreditCardPayment {
    pay(amount) {
        console.log(`信用卡支付: ${amount}`);
    }
}

class PayPalPayment {
    pay(amount) {
        console.log(`PayPal支付: ${amount}`);
    }
}

const payment = new PaymentContext(new CreditCardPayment());
payment.pay(100);
```

3. 命令模式 (Command)

```
class Command {
    execute() {}
}

class LightOnCommand extends Command {
    constructor(light) {
        super();
        this.light = light;
    }

    execute() {
        this.light.turnOn();
    }
}

class RemoteControl {
    constructor() {
        this.commands = [];
    }

    addCommand(command) {
        this.commands.push(command);
    }

    executeCommands() {
        this.commands.forEach(cmd => cmd.execute());
    }
}
```

4. 迭代器模式 (Iterator)

```
class Iterator {
    constructor(collection) {
        this.collection = collection;
        this.index = 0;
    }
```

```
    hasNext() {
        return this.index < this.collection.length;
    }

    next() {
        return this.collection[this.index++];
    }
}

const iterator = new Iterator([1, 2, 3]);
while (iterator.hasNext()) {
    console.log(iterator.next());
}
```

5. 模块模式 (Module Pattern)

```
const Counter = (function() {
    let count = 0;

    return {
        increment() {
            count++;
            console.log(count);
        },
        decrement() {
            count--;
            console.log(count);
        },
        getCount() {
            return count;
        }
    };
})();

Counter.increment(); // 1
Counter.increment(); // 2
console.log(Counter.getCount()); // 2
```

四、JavaScript 特有模式

1. 模块模式 (Module Pattern)

```
const MyModule = (function() {
    let privateVar = '私有变量';

    function privateMethod() {
        console.log('私有方法');
    }

    return {
        publicMethod() {
            privateMethod();
        },
```

```
        publicVar: '公共变量'
    };
})();
```

2. Mixin 模式

```
const sayHi = {
    sayHello() {
        console.log(`Hi, I'm ${this.name}`);
    }
};

class User {
    constructor(name) {
        this.name = name;
    }
}

Object.assign(User.prototype, sayHi);
const user = new User('张三');
user.sayHello();
```

3. 发布-订阅模式

```
class PubSub {
    constructor() {
        this.subscribers = {};
    }

    subscribe(event, callback) {
        if (!this.subscribers[event]) {
            this.subscribers[event] = [];
        }
        this.subscribers[event].push(callback);
    }

    publish(event, data) {
        if (this.subscribers[event]) {
            this.subscribers[event].forEach(callback => callback(data));
        }
    }
}

const pubsub = new PubSub();
pubsub.subscribe('login', (user) => console.log(`${user} 已登录`));
pubsub.publish('login', '张三');
```

五、模式分类汇总

类型	模式
创建型	单例模式、工厂模式、建造者模式、原型模式
结构型	适配器模式、装饰器模式、代理模式、组合模式、外观模式
行为型	观察者模式、策略模式、命令模式、迭代器模式
JS特有	模块模式、Mixin模式、发布-订阅模式

六、Java vs JavaScript 设计模式对比

特性	Java	JavaScript
单例实现	类 + 私有构造函数	闭包 / Proxy
继承	类继承	原型链 / Object.assign
模块化	包 / JAR	ES6模块 / 闭包
接口	显式接口	隐式协议
装饰器	装饰器注解	高阶函数